



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de carrera: Ingeniería en Informática.

Unidad Curricular: Lenguajes y compiladores

Semestre 7 – Sección: I

ANALISIS LEXICO

Profesor:

Marquez, Félix.

Autores:

Longart, Daniela. V-31.445.710.

Martinez, Fabiana. V-30.498.516.

Ortiz, Sebastian. V-30.576.350.

Valencia, Haddan. V-31.818.222.

Ciudad Guayana, Julio del 2026.

RESUMEN ACADEMICO

El presente documento, correspondiente al Tema 4 de la cátedra Lenguaje y Compiladores, explora el proceso de **análisis léxico** (o tokenización), definido como la fase fundamental en la que un compilador transforma una secuencia de caracteres de un programa fuente en una serie de tokens con significado lógico, tales como identificadores, operadores y palabras reservadas. Este mecanismo es esencial para verificar la validez de las estructuras léxicas según las reglas de un lenguaje formal antes de avanzar hacia las etapas de análisis sintáctico y semántico.

El contenido se fundamenta teóricamente en la **jerarquía de Chomsky**, específicamente en el uso de autómatas finitos (AFD) y expresiones regulares para el reconocimiento de lenguajes regulares. El informe detalla dos enfoques metodológicos para la construcción de analizadores léxicos:

- **Implementación desde cero:** Mediante el desarrollo manual de un autómata finito y su codificación en un lenguaje de programación como Python, lo cual permite un control total sobre el proceso y la gestión de errores.
- **Implementación mediante metacompiladores:** Utilizando herramientas de automatización como **Flex**, que facilitan la creación de analizadores a partir de una descripción formal del lenguaje.

El material expone la aplicación práctica de estos conceptos mediante el análisis de un archivo de configuración de red (interfaces), comparando la flexibilidad del desarrollo manual frente a la eficiencia de las herramientas de metacompilación. Finalmente, el documento establece actividades prácticas que exigen el diseño de autómatas, la construcción de lexers mediante expresiones regulares y la investigación sobre el uso de estas tecnologías en áreas como la seguridad informática.

INTRODUCCION

El análisis léxico constituye la fase inicial y esencial en el diseño y construcción de cualquier compilador, siendo su función principal la transformación de una secuencia de caracteres, que conforman el programa fuente, en una secuencia de unidades con significado lógico denominadas tokens. Este proceso, denominado tokenización, permite validar que la estructura del código fuente se ajuste a las reglas léxicas de un lenguaje formal antes de avanzar a las etapas de análisis sintáctico y semántico.

Desde una perspectiva teórica, el analizador léxico o lexer fundamenta su funcionamiento en la teoría de autómatas finitos y el uso de expresiones regulares, los cuales permiten reconocer y validar los patrones léxicos definidos para un lenguaje específico dentro de la jerarquía de Chomsky.

El presente informe aborda el diseño e implementación de analizadores léxicos mediante dos enfoques metodológicos: Implementación desde cero: Basada en la construcción manual de autómatas finitos determinísticos (AFD), lo que otorga un control detallado sobre el proceso de lectura y validación carácter a carácter.

Uso de herramientas de metacompilación: Utilizando tecnologías como Flex para automatizar la generación del analizador a partir de descripciones formales, lo cual favorece una creación más ágil, aunque requiere un entendimiento profundo de la herramienta.

A lo largo de este documento, se explorarán los fundamentos teóricos necesarios, la práctica de implementación en lenguajes de programación como Python y la aplicación de herramientas de automatización, con el objetivo de demostrar cómo se materializan los conceptos de máquinas teóricas en software ejecutable capaz de identificar tanto componentes léxicos válidos como errores en la entrada.

DESARROLLO

Actividad 1: Autómatas de Pila

Los autómatas de pila (PDA, por sus siglas en inglés *Pushdown Automata*) son abstracciones matemáticas fundamentales en la teoría de la computación y de los lenguajes formales. Constituyen una extensión de los autómatas finitos mediante la incorporación de una estructura de almacenamiento basada en el principio LIFO (*Last-In-First-Out*, último en entrar, primero en salir), conocida como pila (Hoogeboom y Engelfriet, 2004). Esta memoria externa adicional les otorga la capacidad de gestionar estructuras anidadas y recursividad, siendo la contraparte computacional exacta de las gramáticas libres de contexto (Galicia Haro y Gelbukh, 2007, junto con Hoogeboom y Engelfriet, 2004).

Un autómata de pila se define formalmente como una tupla matemática de siete elementos (Hoogeboom y Engelfriet, 2004; Nguyen, 2009). La definición exacta se establece como $A = (Q, \Sigma, \Gamma, Z_0, \Delta, q_0, F)$, donde cada componente representa lo siguiente:

- Q : un conjunto finito de estados de control.
- Σ : el alfabeto de entrada finito (los símbolos que lee el autómata de la cinta).
- Γ : El alfabeto finito de la pila (los símbolos que se pueden almacenar en la memoria LIFO).
- Z_0 : el símbolo inicial de la pila que se encuentra en el fondo antes de iniciar cualquier operación.
- Δ : la relación de transición, que es un subconjunto finito de reglas que determinan cómo el autómata cambia de estado y modifica la pila.
- q_0 : el estado inicial de control.

- F : es un subconjunto de Q que representa los estados finales o de aceptación.

Una configuración o descripción instantánea del autómata en un momento dado se describe mediante el estado actual, el contenido no procesado de la cinta de entrada y el contenido actual de la pila (Nguyen, 2009).

Formas de Aceptación

Los autómatas de pila cuentan con dos mecanismos o criterios principales para determinar si una cadena de entrada es válida o aceptada por el lenguaje que modelan (Hoogeboom y Engelfriet, 2004; Nguyen, 2009):

1. Aceptación por estado final: el PDA acepta la cadena de entrada si, tras leerla por completo, la computación se detiene en alguno de los estados que pertenecen al conjunto de estados de aceptación F .
2. Aceptación por pila vacía: el PDA acepta la cadena si, después de leer toda la entrada, el autómata ha vaciado por completo su pila (retirando incluso el símbolo inicial Z_0). En este caso, el estado en el que finalice la máquina es irrelevante.

Ambas formas de aceptación son intercambiables en cuanto a su poder expresivo; mediante algoritmos específicos, es posible construir un PDA que acepte por pila vacía a partir de uno que acepte por estado final y viceversa, reconociendo el mismo lenguaje libre de contexto (Hoogeboom y Engelfriet, 2004).

Operaciones de la Pila

Las transiciones de un PDA (definidas en el conjunto Δ) están condicionadas por tres factores: el estado de control actual, el siguiente símbolo a leer en la cinta de entrada (o

la cadena vacía ϵ o λ , permitiendo movimientos espontáneos sin consumir entrada) y el símbolo que se encuentra en el tope de la pila (Hoogeboom y Engelfriet, 2004).

Al ejecutar una regla de transición, el autómata realiza las siguientes operaciones simultáneas (Nguyen, 2009):

- Pop (Extraer): El símbolo que se encuentra en la cima de la pila es leído y retirado temporalmente.
- Push (Insertar): Tras retirar el símbolo tope, el autómata introduce una nueva cadena de símbolos en el tope de la pila.

Existen casos específicos en la operación: si la cadena insertada es de longitud 2, la operación actúa como un *push* real añadiendo un nuevo elemento sobre el anterior; si la cadena es de longitud 1, es una operación interna de sustitución de la cima; y si la cadena es vacía, la operación actúa como un *pop* estricto, reduciendo la altura de la pila (Nguyen, 2009).

Comparación de Poder Computacional

Existe una diferencia abismal en el poder expresivo entre los Autómatas Finitos (AF) y los Autómatas de Pila (PDA). Los Autómatas Finitos representan el nivel más restringido de la jerarquía de Chomsky (Tipo 3 o lenguajes regulares) y su principal limitación es la ausencia de una memoria auxiliar estructurada (Sarabia Martínez y Torres Samperio, 2026). Esta incapacidad les impide procesar conteos arbitrarios o reconocer lenguajes que presenten dependencias anidadas, debido a que no tienen cómo recordar el número exacto de elementos procesados anteriormente.

Por el contrario, los Autómatas de Pila solucionan esta deficiencia incorporando memoria ilimitada (su pila LIFO). Esto expande su poder computacional para reconocer exitosamente las gramáticas y lenguajes libres de contexto (Tipo 2), abarcando

construcciones con recursividad profunda, emparejamiento de paréntesis y las dependencias anidadas (Sarabia Martínez y Torres Samperio, 2026).

Importancia en el diseño de compiladores

En el área de la informática, los autómatas de pila son la estructura esencial para la construcción de lenguajes de programación y compiladores. Los lenguajes modernos utilizan el control de flujo estructurado y la invocación de módulos o procedimientos de manera anidada y recursiva; un comportamiento que se modela a la perfección con la pila de un PDA (Nguyen, 2009).

Durante la fase de análisis sintáctico (o *parsing*) en el diseño de un compilador, los PDAs funcionan como el motor que valida la gramática del código fuente. Métodos algorítmicos fundamentales para los compiladores trabajan simulando estos autómatas, como el análisis ascendente *Shift-Reduce* o la construcción teórica de analizadores predictivos LR(k), los cuales aplican transiciones desplazando símbolos a la pila (*shift*) y agrupándolos sintácticamente en estructuras de árbol (*reduce*) basados en la entrada y los estados de la pila (Hoogeboom y Engelfriet, 2004). De este modo, los PDA garantizan la robustez en la traducción y validación del código antes de su ejecución.

Ejemplo de un autómata de pila

```
def parser_sintactico_expresion(expresion):  
    """  
    Simula un Analizador Sintáctico basado en Pila para validar  
    el balanceo y anidamiento correcto de paréntesis en expresiones  
    matemáticas.  
    """  
    pila = [] # Pila LIFO auxiliar para controlar el anidamiento  
  
    print(f"Analizando expresión: '{expresion}'")  
  
    for token in expresion:
```

```

        # Operación SHIFT simulada: Si encontramos apertura, la enviamos a la
pila
        if token in ['(', '[', '{']:
            pila.append(token)
            print(f"Token: '{token}' -> SHIFT (Guardar en pila) | Pila actual:
{pila}")

        # Operación REDUCE simulada: Si encontramos cierre, verificamos
correspondencia y hacemos POP
        elif token in [')', ']', '}']:
            if not pila:
                print(f"Error: Se encontró un cierre '{token}' pero la pila
está vacía.")
                return False

            tope = pila[-1]
            # Validación de correspondencia sintáctica entre pares abiertos y
cerrados
            if (token == ')' and tope == '(') or \
                (token == ']' and tope == '[') or \
                (token == '}' and tope == '{'):
                pila.pop()
                print(f"Token: '{token}' -> REDUCE (Emparejado con '{tope}') |
Pila actual: {pila}")
            else:
                print(f"Error: Discordancia sintáctica entre el tope '{tope}' y
el cierre '{token}'.")
                return False

        # Verificación final de pila limpia (Aceptación)
        if not pila:
            print("Resultado sintáctico: EXPRESIÓN VÁLIDA Y BALANCEADA.\n")
            return True
        else:
            print(f"Resultado sintáctico: EXPRESIÓN INVÁLIDA. Faltaron cerrar
elementos: {pila}\n")
            return False

# --- Ejecución del ejemplo ---
print("=== CASO 1: Expresión sintácticamente correcta ===")
parser_sintactico_expresion("{2 * [5 + (3 - 1)]}")

print("=*50 + "\n")

print("=== CASO 2: Expresión sintácticamente incorrecta ===")
parser_sintactico_expresion("{2 * [5 + (3 - 1)]}")

```


El script recorre la expresión matemática de izquierda a derecha y toma decisiones basadas únicamente en lo que ve y en lo que hay en el tope de la pila:

1. Guardar (*shift*): cada vez que el programa lee un símbolo de apertura ((, [, {), lo "desplaza" (hace un *Shift*) y lo guarda en la parte superior de la pila.
2. Operación comprobar y eliminar (*reduce*): cuando el programa se topa con un símbolo de cierre (,], }), mira inmediatamente qué hay en el tope de la pila. Si el símbolo de cierre coincide con el tipo de apertura que estaba arriba del todo, significa que esa "caja" se cerró correctamente. Entonces, el programa saca ese plato de la pila (hace un *Reduce* o *Pop*).
 - o Si el programa ve un cierre pero la pila está vacía, o si ve un cierre que no coincide con el plato de arriba (por ejemplo, intentar cerrar un corchete] con un paréntesis)), el inspector detecta un error de sintaxis de inmediato y detiene el programa.
3. Condición de aceptación: una vez que el programa termina de leer toda la expresión, revisa la pila. Si la pila quedó completamente vacía, significa que cada símbolo que se abrió encontró a su pareja exacta en el orden correcto. La expresión es válida. Si queda algún símbolo dentro, significa que algo se quedó abierto y el código es inválido.

Construcción de un Analizador Léxico para un Subconjunto de Rust

(Lenguaje L)

1. Manual de Usuario: Flex (Metacompilador)

Flex (Fast Lexical Analyzer Generator) es un metacompilador especializado en la generación de analizadores léxicos. A diferencia de un compilador convencional que traduce código fuente a lenguaje máquina, Flex actúa como un "generador de programas":

- **Función:** Recibe como entrada un archivo de especificación (.l) que contiene reglas basadas en expresiones regulares.
- **Proceso:** A partir de estas reglas, Flex genera automáticamente un archivo de código fuente en lenguaje C (lex.yy.c) que contiene la lógica necesaria para leer texto de entrada y segmentarlo en unidades lógicas llamadas **tokens**.
- **Uso:** El programador define qué constituye un token (por ejemplo, "un número es cualquier dígito del 0 al 9") y Flex se encarga de crear el motor de búsqueda y clasificación. Es la herramienta estándar para la fase inicial de análisis en la teoría de compiladores.

2. Descripción del Lenguaje L

El lenguaje **L** es un subconjunto minimalista inspirado en el lenguaje **Rust**. Su objetivo es permitir la declaración y asignación básica de variables, así como el uso de estructuras de control fundamentales.

Especificación de Tokens del Lenguaje L:

- **Palabras clave:** let (declaración), fn (funciones), if (condicional).
- **Identificadores:** Secuencias alfanuméricas que comienzan con letra o guion bajo.
- **Literales numéricos:** Secuencias de dígitos (0-9).
- **Operadores:** = (asignación).
- **Delimitadores:** ; (punto y coma).
- **Ignorados:** Espacios, tabulaciones y saltos de línea.

3. Proceso de Creación e Implementación

Entorno de Desarrollo

Para la implementación, se utilizó **MSYS2** como plataforma de compatibilidad POSIX sobre Windows, lo que permitió instalar y ejecutar herramientas nativas de sistemas UNIX.

- **Metacompilador:** Flex 2.6.4.
- **Compilador C:** GCC 16.1.0 (MinGW-w64).
- **Automatización:** Make 4.4.

Pasos de Implementación

1. **Configuración del Entorno:** Se instalaron las herramientas mediante el gestor de paquetes de MSYS2 (pacman):

```
Bash
pacman -S mingw-w64-ucrt-x86_64-gcc flex make
```

2. **Definición Léxica (Lexl.l):** Se crearon las reglas de expresiones regulares. Se incluyó la directiva %option noyywrap para gestionar la finalización del archivo sin dependencias externas de librerías.

3. **Automatización (Makefile):** Se definió un archivo Makefile para agilizar la compilación:

- a. **Comando make:** Ejecuta flex para generar el código C y gcc para compilar el ejecutable.
- b. **Comando make clean:** Elimina los archivos temporales generados (lex.yy.c y el binario).

4. **Compilación y Ejecución:**

- a. Se ejecutó make en la terminal, generando el binario Lexl.exe.
- b. Se probó el lexer mediante redirección de entrada: ./Lexl < archivo_prueba.rs.

Anexo: Código Fuente (Resumen)

archivo Lexl.l

Fragmento de código

```
%{  
  
#include <stdio.h>  
  
%}  
  
%option noyywrap  
  
%%  
  
"let"          { printf("TOKEN_LET\n"); }  
"fn"           { printf("TOKEN_FN\n"); }  
[0-9]+         { printf("TOKEN_NUMERO(%s)\n", yytext); }  
[a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN_ID(%s)\n", yytext); }  
"="            { printf("TOKEN_ASIGNACION\n"); }  
";"            { printf("TOKEN_PUNTO_COMA\n"); }  
[ \t\n]+       { /* Ignorar */ }  
.  
               { printf("TOKEN_DESCONOCIDO(%s)\n", yytext); }  
  
%%  
  
int main() { yylex(); return 0; }
```

archivo Makefile

Makefile

```
all:  
  
    flex Lex1.l  
  
    gcc lex.yy.c -o Lex1  
  
clean:  
  
    rm -f lex.yy.c Lex1.exe
```

Ejecucion, Instalacion y creacion del proyecto:

```
MINGW64:/c/Users/said y alida/OneDrive/Documentos/Tema 4
cargo init --name lenguaje_1
    Creating binary (application) package
note: see more 'Cargo.toml' keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

said y alida@DESKTOP-COFS8LU MINGW64 ~/OneDrive/Documentos/Tema 4 (master)
$ cargo add pest
    Updating crates.io index
    Adding pest v2.8.7 to dependencies
    Features:
    + memchr
    + std
    - const_prec_climber
    - miette-error
    - pretty-print
    Updating crates.io index
    Locking 3 packages to latest Rust 1.96.0 compatible versions

said y alida@DESKTOP-COFS8LU MINGW64 ~/OneDrive/Documentos/Tema 4 (master)
$ cargo add pest_derive
    Updating crates.io index
    Adding pest_derive v2.8.7 to dependencies
    Features:
    + std
    - grammar-extras
    Updating crates.io index
    Locking 7 packages to latest Rust 1.96.0 compatible versions
    Adding pest_derive v2.8.7
    Adding pest_generator v2.8.7
    Adding pest_meta v2.8.7
    Adding proc-macro2 v1.0.106
    Adding quote v1.0.46
    Adding syn v2.0.118
    Adding unicode-ident v1.0.24

said y alida@DESKTOP-COFS8LU MINGW64 ~/OneDrive/Documentos/Tema 4 (master)
$ ..
```

Ejecucion del comando make en MSYS2:

```
M /c/Users/said y alida/OneDrive/Documentos/Tema 4
said y alida@DESKTOP-COFS8LU UCRT64 ~
$ cd "/c/Users/said y alida/OneDrive/Documentos/Tema 4"

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ ls
Cargo.lock Cargo.toml Lexer_Docker.py Lex1.1 Makefile __pycache__ src target

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ make
flex Lex1.1
gcc lex.yy.c -o Lex1 -lfl
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/16.1.0/././././././x86_64-w64-mingw32/bin/ld.exe:
cannot find -lfl: No such file or directory
collect2.exe: error: ld returned 1 exit status
make: *** [Makefile:3: all] Error 1

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ all:
flex Lex1.1
gcc lex.yy.c -o Lex1
-bash: all:: command not found
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/16.1.0/././././././x86_64-w64-mingw32/bin/ld.exe:
C:/msys64/tmp/ccExILfw.o:lex.yy.c:(.text+0x6ea): undefined reference to 'yywrap'
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/16.1.0/././././././x86_64-w64-mingw32/bin/ld.exe:
C:/msys64/tmp/ccExILfw.o:lex.yy.c:(.text+0x130d): undefined reference to 'yywrap'
collect2.exe: error: ld returned 1 exit status
```

Ejecucion exitosa:

```
M /c/Users/said y alida/OneDrive/Documentos/Tema 4
-bash: all:: command not found
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/16.1.0/././././././x86_64-w64-mingw32/bin/ld.exe:
C:/msys64/tmp/ccExILfw.o:lex.yy.c:(.text+0x6ea): undefined reference to 'yywrap'
C:/msys64/ucrt64/bin/./lib/gcc/x86_64-w64-mingw32/16.1.0/././././././x86_64-w64-mingw32/bin/ld.exe:
C:/msys64/tmp/ccExILfw.o:lex.yy.c:(.text+0x130d): undefined reference to 'yywrap'
collect2.exe: error: ld returned 1 exit status

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ make clean
rm -f lex.yy.c Lex1.exe

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ make
flex Lex1.1
gcc lex.yy.c -o Lex1

said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ echo "let x = 10;" > test.txt
./Lex1 < test.txt
TOKEN_LET
TOKEN_ID(x)
TOKEN_ASSIGNACION
TOKEN_NUMERO(10)
TOKEN_PUNTO_COMA

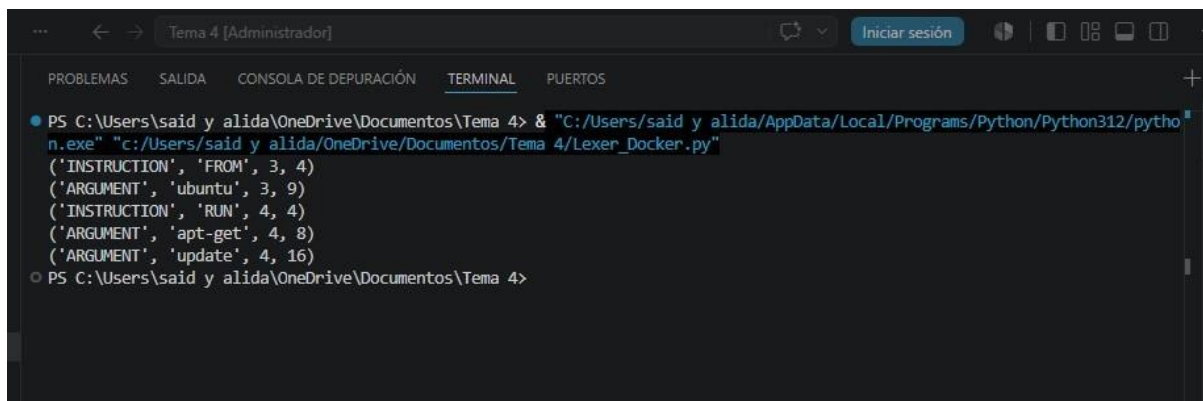
said y alida@DESKTOP-COFS8LU UCRT64 /c/Users/said y alida/OneDrive/Documentos/Tema 4
$ !
```

Explicación del Código Lexer_Docker.py

Este script implementa un analizador léxico (lexer) diseñado para procesar archivos de configuración tipo Dockerfile. El funcionamiento se basa en la técnica de reconocimiento de patrones mediante expresiones regulares aplicada al procesamiento de cadenas de texto.

1. **Definición del Alfabeto (Tokens):** El código utiliza una lista de tuplas tokens donde cada elemento empareja un nombre descriptivo con una expresión regular (Regex). Esto permite clasificar el texto entrante en categorías lógicas como instrucciones de Docker (FROM, RUN), argumentos, comentarios o caracteres no reconocidos.
2. **Motor del Lexer (lexer function):** Compilación de Patrones: Utiliza `re.finditer` para buscar coincidencias de los tokens definidos a lo largo de toda la cadena de entrada de manera eficiente.
 - a. Captura de Grupos Nombrados: Emplea la sintaxis `(P<name>...)` de Python para identificar automáticamente qué tipo de token se ha detectado en cada iteración.
 - b. Gestión de Estado: Mantiene un seguimiento del número de línea (`line_num`) y la posición del cursor (`line_start`) para proporcionar reportes precisos en caso de errores léxicos.
3. **Flujo de Datos:** El lexer funciona como un generador (usando `yield`), lo que optimiza el uso de memoria al procesar el archivo token por token en lugar de cargar toda la lista en memoria.
4. **Manejo de Errores:** Incluye una categoría `MISMATCH` que captura cualquier carácter no definido en la gramática, disparando una excepción `RuntimeError` que detiene el análisis ante una sintaxis inválida.

Ejecucion exitosa del codigo python:



```
PS C:\Users\said y alida\OneDrive\Documentos\Tema 4> & "C:/Users/said y alida/AppData/Local/Programs/Python/Python312/python.exe" "C:/Users/said y alida/OneDrive/Documentos/Tema 4/Lexer_Docker.py"
('INSTRUCTION', 'FROM', 3, 4)
('ARGUMENT', 'ubuntu', 3, 9)
('INSTRUCTION', 'RUN', 4, 4)
('ARGUMENT', 'apt-get', 4, 8)
('ARGUMENT', 'update', 4, 16)
PS C:\Users\said y alida\OneDrive\Documentos\Tema 4>
```

Actividad 4: Aplicación de Analizadores Léxicos en Seguridad Informática

Indagación y Reflexión

En el ámbito de la ciberseguridad, el procesamiento masivo, rápido y preciso de texto es una necesidad crítica. Los analistas de seguridad y los sistemas automatizados (como firewalls, IDS/IPS y motores antivirus) leen constantemente archivos de configuración, logs de red y firmas de malware. En este contexto, un metacompilador como Flex es una herramienta sumamente poderosa, ya que permite generar analizadores léxicos en C altamente optimizados capaces de escanear millones de líneas de texto en fracciones de segundo para detectar patrones anómalos o maliciosos.

Un área específica donde Flex tiene una aplicación directa es en el parseo de lenguajes de reglas de detección de amenazas. Si una regla de seguridad está mal escrita, el motor podría fallar o generar falsos positivos. El analizador léxico asegura que las reglas escritas por los analistas cumplan estrictamente con la gramática formal antes de ser evaluadas.

Lenguaje Propuesto: YARA

YARA es un lenguaje ampliamente utilizado en la comunidad de InfoSec para la identificación y clasificación de muestras de malware. Permite a los analistas crear descripciones (reglas) basadas en patrones de texto o binarios.

Una regla de YARA típica tiene la siguiente estructura:


```

1
2
3  rule Deteccion_Malware_Basico {
4  |   meta:
5  |       author = "Equipo de Seguridad"
6  |       description = "Detecta un string sospechoso"
7  |   strings:
8  |       $cadena_hex = { E2 34 ?? C8 A? FB }
9  |       $cadena_texto = "cmd.exe /c format c:"
10 |   condition:
11 |       $cadena_hex or $cadena_texto
12 }
13

```

Tokenización del Lenguaje YARA usando FLEX

Si construyéramos un analizador léxico para YARA utilizando Flex, la especificación de tokens (el archivo .l) definiría los componentes léxicos básicos del lenguaje de la siguiente manera:

1. **Palabras Reservadas (Keywords):** Reglas directas para identificar la estructura principal.
 - a. rule TOKEN_RULE
 - b. meta TOKEN_META
 - c. strings TOKEN_STRINGS
 - d. condition TOKEN_CONDITION
2. **Operadores y Lógica:**
 - a. and, or, not TOKEN_OPERADOR_LOGICO
 - b. =, ==, >=, <= TOKEN_OPERADOR_RELACIONAL
3. **Identificadores:**
 - a. Nombres de reglas (ej. Deteccion_Malware_Basico): Expresión regular
 [a-zA-Z_][a-zA-Z0-9_]* TOKEN_IDENTIFICADOR

- b. Variables de cadenas (ej. cadena_hex): Expresión regular `\$[a-zA-Z0-9_]+` TOKEN_VARIABLE_STRING

4. Literales y Valores:

- a. Cadenas de texto (ej. "cmd.exe"): Expresión regular `\\"[^\"]*" \`
TOKEN_STRING_TEXTO
- b. Cadenas Hexadecimales (ej. { E2 34 }): Expresión regular `\{[a-fA-F0-9s]+\}` TOKEN_STRING_HEX

5. Delimitadores:

- a. {, }, : TOKEN_LLAVE_ABRE, TOKEN_LLAVE_CIERRA,
TOKEN_DOS_PUNTOS

6. Ignorados:

- a. Espacios, tabulaciones y comentarios (como // comentario) serían ignorados por el analizador para no generar tokens innecesarios.

Mediante esta tokenización, el lexer generado por Flex procesaría el archivo YARA y pasaría una secuencia limpia de tokens al analizador sintáctico, garantizando que el analista de seguridad no cometió errores tipográficos (como escribir stirngs en lugar de strings) antes de ejecutar el motor de búsqueda en un sistema infectado.

CONCLUSION

El análisis léxico y sintáctico representa la piedra angular en la arquitectura de los compiladores, funcionando como el mecanismo indispensable para transformar secuencias de caracteres en unidades lógicas con significado. A través de la integración de fundamentos teóricos como la jerarquía de Chomsky, los autómatas finitos y los autómatas de pila se establece un marco sólido que permite validar estructuras gramaticales complejas mediante el uso de memoria auxiliar LIFO y expresiones regulares.

La implementación de estas técnicas demuestra que existe una sinergia necesaria entre el control granular que ofrece el desarrollo manual y la eficiencia operativa proporcionada por metacompiladores como Flex. Esta versatilidad metodológica permite no solo la creación de lenguajes personalizados, sino también la optimización de procesos críticos en entornos de software y seguridad informática.

En este último ámbito, el uso de analizadores léxicos para validar lenguajes como YARA resulta determinante, ya que garantiza que los sistemas de detección de amenazas operen sobre reglas sintácticamente íntegras, minimizando así el margen de error en la identificación de malware.

En conjunto, la convergencia entre la teoría de lenguajes formales y la aplicación práctica mediante herramientas de automatización es lo que permite materializar modelos abstractos en soluciones tecnológicas robustas, capaces de afrontar los retos actuales en el desarrollo de software y la protección de sistemas.

REFERENCIAS BIBLIOGRAFICAS

- Levine, J. R. (2009). *Flex & Bison* (1ra ed.). O'Reilly Media.
- Klabnik, S., y Nichols, C. (2023). *The Rust Programming Language* (2da ed.). No Starch Press. <https://doc.rust-lang.org/book/>
- Python Software Foundation. (2026). *re — Regular expression operations*. Python 3.12.4 Documentation. <https://docs.python.org/3/library/re.html>
- MSYS2 Project. (2026). *MSYS2: Software distribution and building platform for Windows*. <https://www.msys2.org/>
- Galicia Haro, S. N., & Gelbukh, A. (2007). *Investigaciones en análisis sintáctico para el español*. Instituto Politécnico Nacional. <https://www.gelbukh.com/libro-investigaciones/LibroSint.pdf>
- Hoogeboom, H. J., & Engelfriet, J. (2004). Pushdown Automata. En C. Martín-Vide, V. Mitrana, & G. Paun (Eds.), *Formal Languages and Applications* (pp. 117-138). Springer. https://www.researchgate.net/publication/242427937_Pushdown_Automata
- Nguyen, V. T. (2009). *Pushdown Automata and Inclusion Problems* [Tesis doctoral, Japan Advanced Institute of Science and Technology]. <http://www.jaist.ac.jp/~mizuhito/thesis/NguyenVanTang.pdf>

- Sarabia Martínez, A. Y., & Torres Samperio, G. A. (2026). Autómatas finitos y reconocimiento de lenguajes formales. *ASCE MAGAZINE*, 5(2), 453–472.
<https://dialnet.unirioja.es/descarga/articulo/10764098.pdf>
- Sikorski, M., y Honig, A. (2012). *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press.
- Alvarez, V. (2026). *YARA Documentation*. Recuperado de <https://yara.readthedocs.io/>